Claude Certified Architect

Foundations Exam Study Guide

Magnus Rodseth

29.03.2026

Contents

Agentic Architecture & Orchestration (27% of Exam) 7
1. The Agentic Loop Lifecycle . . . 7
Loop Steps . . . 7
Code Example: Basic Agentic Loop . . . 7
Anti-Patterns in the Agentic Loop . . . 8
2. Model-Driven vs Pre-Configured Decision Trees . . . 8
Model-Driven Architecture . . . 8
Pre-Configured Decision Trees . . . 8
3. Hub-and-Spoke Coordinator-Subagent Architecture . . . 9
Architecture Overview . . . 9
Key Properties . . . 9
Why Isolated Context Matters . . . 9
4. Task Tool for Spawning Subagents . . . 10
Configuration Requirements . . . 10
AgentDefinition Configuration . . . 10
Parallel Subagent Spawning . . . 10
5. fork_session for Parallel Exploration . . . 10
Use Cases . . . 11
Properties . . . 11
6. Programmatic Enforcement vs Prompt-Based Guidance . . . 11
Programmatic Enforcement (Hooks) . . . 11
Prompt-Based Guidance . . . 12
PostToolUse Hooks for Compliance . . . 12
Summary Table . . . 12
7. Structured Handoff Protocols . . . 12
Escalation Payload Example . . . 12
Why Structured Handoffs Matter . . . 13
8. Task Decomposition Patterns . . . 13
Prompt Chaining (Static Decomposition) . . . 13
Dynamic Adaptive Decomposition . . . 13
9. Multi-Pass Review Pattern . . . 13
Structure . . . 13
Why Two Passes? . . . 14
Implementation . . . 14
10. Session Management . . . 14
–resume . . . 14
fork_session . . . 14
update_context . . . 14
Scratchpad Pattern . . . 14
11. Workflow Patterns . . . 15
Parallelization . . . 15

Chaining . . . 15
Routing . . . 15
Evaluator-Optimizer . . . 16
12. Agents vs Workflows . . . 16
13. Agent Design Principles . . . 16
Small Set of Abstract Tools . . . 16
Environment Inspection After Actions . . . 17
14. Model Selection . . . 17
Selection Guidelines . . . 17

Tool Design & MCP Integration (18% of Exam) 18
1. Tool Descriptions: The Primary Selection Mechanism . . . 18
What to Include in Tool Descriptions . . . 18
Example: Well-Designed Tool Description . . . 18
Splitting Broad Tools into Granular Tools . . . 18
2. Structured Error Responses . . . 19
Error Response Schema . . . 19
Error Categories . . . 19
Key Rules . . . 20
Why This Matters . . . 20
3. Scoped Tool Access . . . 20
Principle: 4-5 Tools Per Agent . . . 20
Example Scoping . . . 20
4. tool_choice Parameter . . . 21
Code Example . . . 21
5. MCP (Model Context Protocol) . . . 22
Server Scoping . . . 22
Environment Variable Expansion . . . 22
MCP Primitives . . . 22
MCP Transports . . . 23
MCP Sampling . . . 24
6. Built-in Tools in Claude Code . . . 24
Edit Fallback Pattern . . . 24
7. Batch Tool Pattern . . . 25
8. Built-in API Tools . . . 25
Web Search Tool . . . 25
Text Editor Tool . . . 25
Computer Use . . . 26
9. Tool Design Principles Summary . . . 26

Claude Code Configuration & Workflows (20% of Exam) 27
1. CLAUDE.md Hierarchy . . . 27
Levels . . . 27

Key Points . . . 27
@import Syntax . . . 27
2. .claude/rules/ Directory . . . 28
Structure . . . 28
YAML Frontmatter with Glob Patterns . . . 28
3. Custom Slash Commands . . . 29
Project Commands . . . 29
Personal Commands . . . 29
4. Skills: SKILL.md . . . 29
Frontmatter Options . . . 29
5. Plan Mode vs Direct Execution . . . 30
Plan Mode . . . 30
Direct Execution . . . 30
Plan Mode vs Thinking Mode . . . 30
6. Key Interaction Patterns . . . 31
Explore Subagent . . . 31
Interview Pattern . . . 31
Iterative Refinement . . . 31
7. CI/CD Integration . . . 31
Non-Interactive Mode . . . 31
Structured Output . . . 32
8. Session Context Isolation . . . 32
Self-Review Limitation . . . 32
Providing Existing Test Files . . . 32
9. Claude Code SDK . . . 32
Default Behavior . . . 32
Key Configuration . . . 33
10. GitHub Integration . . . 33
11. /init Command . . . 33
12. Three Memory Types . . . 33
13. Keyboard Controls . . . 34
Compact vs Clear . . . 34
14. Hooks System . . . 34
PreToolUse Hooks . . . 34
PostToolUse Hooks . . . 35
Hook Pattern: TypeScript Type Checker . . . 35
Hook Pattern: Duplicate Code Prevention . . . 35
15. Configuration Quick Reference . . . 36

Prompt Engineering & Structured Output (20% of Exam) 37
1. Explicit Criteria Over Vague Instructions . . . 37
Bad: Vague . . . 37
Good: Explicit . . . 37

Why This Matters . . . 37
2. Few-Shot Examples . . . 37
Purposes . . . 37
Reducing Hallucination . . . 37
Standardized Formats . . . 38
3. tool_use with JSON Schemas . . . 39
What Schemas Solve . . . 39
Schema Example . . . 39
tool_choice Options (Recap) . . . 39
4. Schema Design Patterns . . . 40
Optional (Nullable) Fields . . . 40
Resilient Catch-All Pattern (Extensible Enums) . . . 40
Schema Redundancy for Validation . . . 41
detected_pattern Field . . . 41
5. Retry-with-Error-Feedback . . . 41
When It Works . . . 41
When It Fails . . . 42
6. Message Batches API . . . 42
Key Properties . . . 42
When to Use . . . 42
Batch Frequency Calculation . . . 42
7. Review Patterns . . . 43
Self-Review Limitations . . . 43
Multi-Pass Review . . . 43
Independent Review Instances . . . 43
Confidence-Based Routing . . . 43
8. Temperature . . . 43
9. Response Prefilling + Stop Sequences . . . 44
Prefilling . . . 44
Stop Sequences . . . 44
10. Extended Thinking . . . 45
How It Works . . . 45
When to Use . . . 45
Decision Framework . . . 45
11. System Prompts . . . 45
Purpose . . . 45
Best Practices . . . 45
12. Being Specific: Type A and Type B Guidelines . . . 46
Type A: Attribute Guidelines . . . 46
Type B: Step-by-Step Guidelines . . . 46
13. XML Tags for Content Structure . . . 46
14. Prompt Evaluation Workflow . . . 47
The Cycle . . . 47

Grading Methods . . . 47
15. AI Fluency: The 4 Ds . . . 47
Delegation . . . 47
Description . . . 48
Discernment . . . 48
Diligence . . . 48
16. Human-AI Interaction Modes . . . 48
Key Differences . . . 48

Context Management & Reliability (15% of Exam) 50
1. Tool Context Pruning . . . 50
Why Prune . . . 50
How to Prune . . . 50
Application-Side Filtering . . . 50
2. Long Session Compression . . . 50
Strategy . . . 50
Example . . . 51
Compact Command . . . 51
3. Resuming Async Sessions . . . 51
Why Filter tool_result Messages . . . 51
4. Scratchpad Pattern . . . 51
Structure . . . 52
Why Use a Scratchpad . . . 52
5. Directed Codebase Exploration . . . 52
Strategy . . . 52
Example Flow . . . 53
6. Graceful Tool Failure . . . 53
Rules . . . 53
Error Response Structure . . . 53
Success with Empty Results . . . 53
7. Human-in-the-Loop Calibration . . . 54
Field-Level Confidence . . . 54
Validate Across Segments . . . 54
8. Escalation Patterns . . . 54
Honor Immediate Requests . . . 54
Gather Context for Complex Issues . . . 54
Structured Summary Payload . . . 54
9. API Statelessness . . . 55
Core Principle . . . 55
Implementation . . . 55
Multi-Turn Conversation Management . . . 55
10. Prompt Caching . . . 56
What It Does . . . 56

TTL (Time to Live) . . . 56
Cache Breakpoints . . . 56
Processing Order . . . 56
Caching Strategy . . . 56
11. Response Streaming . . . 57
SSE (Server-Sent Events) . . . 57
Key Event Types . . . 57
content_block_delta . . . 57
12. Citations . . . 58
Types . . . 58
Purpose . . . 58
Usage . . . 58
13. Context Window . . . 58
Varies by Model . . . 59
Managing Context . . . 59
14. RAG Pipeline (Retrieval-Augmented Generation) . . . 59
Full Pipeline . . . 59
Steps . . . 59
15. Chunking Strategies . . . 59
Size-Based Chunking . . . 59
Structure-Based Chunking . . . 60
Semantic-Based Chunking . . . 60
16. Embeddings . . . 60
What They Are . . . 60
Example . . . 60
Cosine Similarity / Distance . . . 61
17. Hybrid Search . . . 61
Components . . . 61
Reciprocal Rank Fusion (RRF) . . . 61
18. Reranking . . . 61
Pipeline with Reranking . . . 62
Why Rerank . . . 62
19. Contextual Retrieval . . . 62
Problem . . . 62
Solution . . . 62
Benefits . . . 62
20. Data Evolution Rule . . . 63
Amended Fields . . . 63
Why This Matters . . . 63
21. Summary of Key Reliability Patterns . . . 63

Agentic Architecture & Orchestration (27% of Exam)

1. The Agentic Loop Lifecycle

The agentic loop is the core execution cycle that powers Claude-based agents. Understanding each phase is critical.

Loop Steps

1. **Send request** to the model with conversation history and available tools
2. **Inspect `stop_reason`** on the response:
   - `"tool_use"`: the model wants to call one or more tools
   - `"end_turn"`: the model considers the task complete
3. **If `tool_use`**: execute the requested tool(s), collect results
4. **Return tool results** as `tool_result` messages appended to conversation history
5. **Loop back to step 1** with the updated conversation

Code Example: Basic Agentic Loop

```
import anthropic

client = anthropic.Anthropic()
messages = [{"role": "user", "content": "Find the largest file in /src"}]

while True:
    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=4096,
        tools=tools,
        messages=messages,
    )

    # Append assistant response to history
    messages.append({"role": "assistant", "content": response.content})

    if response.stop_reason == "end_turn":
        break  # Model is done

    if response.stop_reason == "tool_use":
        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                result = execute_tool(block.name, block.input)
```

```
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result,
                })
        messages.append({"role": "user", "content": tool_results})
```

Anti-Patterns in the Agentic Loop

Anti-Pattern	Why It Fails	Correct Approach
Parsing natural language signals (e.g., “I’m done”)	Unreliable, the model may say “done” mid-task	Use `stop_reason` field exclusively
Arbitrary iteration caps (e.g., `max_loops=10`)	Cuts off legitimate long-running tasks prematurely	Trust the model’s `end_turn` signal
Checking text content for completion keywords	Fragile, language-dependent	Only check `stop_reason`
String-matching tool output for decisions	Breaks on format changes	Let the model interpret tool results

Key principle: The model, not your code, decides when the task is complete. The `stop_reason` is the only reliable signal.

---

2. Model-Driven vs Pre-Configured Decision Trees

Model-Driven Architecture

The model dynamically decides which tools to call, in what order, and when to stop. This is the recommended pattern for agentic systems.

- The model selects tools based on context
- No hardcoded if/else branching in orchestration code
- More flexible, handles novel scenarios
- Relies on good tool descriptions and system prompts

Pre-Configured Decision Trees

Fixed, hand-coded logic dictates the sequence of tool calls and branching.

- Deterministic and predictable
- Brittle when encountering edge cases
- Difficult to maintain as complexity grows
- Appropriate only when the workflow is completely known in advance

Exam insight: The exam favors model-driven approaches. Pre-configured decision trees are generally an anti-pattern in agentic systems.

---

3. Hub-and-Spoke Coordinator-Subagent Architecture

This is the primary multi-agent pattern tested on the exam.

Architecture Overview

```
          ┌──────────────┐
          │ Coordinator  │  (Hub)
          │   Agent      │
          └──────┬───────┘
        ┌────────┼────────┐
        │        │        │
   ┌────▼───┐ ┌──▼─────┐ ┌▼───────┐
   │Sub-    │ │Sub-    │ │Sub-    │  (Spokes)
   │agent A│ │agent B│ │agent C│
   └────────┘ └────────┘ └────────┘
```

Key Properties

- **Coordinator** owns the overall task, delegates subtasks
- **Subagents have isolated context**: they do NOT inherit the coordinator's conversation history
- The coordinator provides explicit instructions when spawning each subagent
- Results flow back to the coordinator, which synthesizes them

Why Isolated Context Matters

- Prevents context window bloat
- Subagents focus on their narrow task without distraction
- Reduces cost (smaller context per subagent call)
- Prevents information leakage between subtasks

---

4. Task Tool for Spawning Subagents

The `Task` tool is the mechanism for spawning subagents in Claude Code.

Configuration Requirements

- `allowedTools` on the coordinator **must include `"Task"`**
- Each subagent is defined via `AgentDefinition`
- The coordinator can spawn multiple subagents in a single response (parallel execution)

AgentDefinition Configuration

```
{
  "name": "code-reviewer",
  "description": "Reviews code for style, bugs, and security issues",
  "systemPrompt": "You are a code review specialist. Focus on...",
  "allowedTools": ["Read", "Grep", "Glob"]
}
```

Key fields:

- **`description`**: tells the coordinator when to use this subagent. This is the primary routing mechanism.
- **`systemPrompt`**: scopes the subagent's behavior and expertise
- **`allowedTools`**: restricts what the subagent can do (principle of least privilege)

Parallel Subagent Spawning

The coordinator can issue multiple `Task` calls in a single response turn:

```
Coordinator response:
  - Task(agent="file-analyzer", input="Analyze src/auth.ts")
  - Task(agent="file-analyzer", input="Analyze src/db.ts")
  - Task(agent="test-writer", input="Write tests for auth module")
```

All three execute concurrently, and results are returned together.

---

5. fork_session for Parallel Exploration

`fork_session` creates parallel branches of a session for exploration without contaminating the main conversation.

Use Cases

- Exploring multiple solution approaches simultaneously
- Testing different strategies before committing
- Parallel investigation of separate concerns

Properties

- Each fork gets its own isolated context
- Forks do not affect the parent session
- Results can be brought back to the parent by the coordinator

---

6. Programmatic Enforcement vs Prompt-Based Guidance

This is a critical distinction for the exam.

Programmatic Enforcement (Hooks)

- **Deterministic guarantees**: always executes, cannot be bypassed
- Implemented via `PreToolUse` and `PostToolUse` hooks
- Example: blocking file writes to production directories

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write",
        "command": "python /scripts/check_write_allowed.py"
      }
    ]
  }
}
```

- `PreToolUse`: runs before a tool executes
  - Exit code 0: allow the tool call
  - Exit code 2: block the tool call
- `PostToolUse`: runs after a tool executes
  - Cannot block (tool already ran)
  - Used for logging, normalization, compliance checks

Prompt-Based Guidance

- **Probabilistic compliance**: usually followed, but not guaranteed
- Implemented via system prompts, CLAUDE.md instructions
- Example: “Always run tests before committing”
- The model may skip instructions under pressure or ambiguity

PostToolUse Hooks for Compliance

Common patterns:

- **Data normalization**: standardize phone numbers, addresses after data retrieval
- **Compliance enforcement**: redact PII from tool outputs before they enter context
- **Audit logging**: record every tool invocation for compliance trails

Summary Table

Property	Hooks (Programmatic)	Prompts (Guidance)
Reliability	Deterministic, 100%	Probabilistic, ~95%
Bypass risk	None (code-enforced)	Model may ignore
Flexibility	Rigid, must be coded	Natural language, easy to add
Use case	Security, compliance	Style, preferences

---

7. Structured Handoff Protocols

When escalating from an agent to a human (or between agents), use structured payloads, not free-form text.

Escalation Payload Example

```
{
  "customer_id": "C-12345",
  "root_cause": "Duplicate charge on order #98765",
  "amount": 49.99,
  "currency": "USD",
  "recommended_action": "Refund full amount",
  "confidence": 0.92,
```

```
  "conversation_summary": "Customer reported duplicate charge...",
  "attempted_resolutions": ["Checked order status", "Verified payment records"]
}
```

Why Structured Handoffs Matter

- Receiving agent (or human) gets all context immediately
- No re-discovery or re-asking for basic information
- Standardized format enables automation on the receiving end
- Audit trail with machine-readable fields

---

8. Task Decomposition Patterns

Prompt Chaining (Static Decomposition)

- Pre-defined sequence of steps
- Output of step N feeds into step N+1
- Works well for known, repeatable workflows
- Example: Extract data, then transform, then load

```
Step 1: Parse PDF → extracted_text
Step 2: Classify(extracted_text) → category
Step 3: Route(category) → appropriate_handler
```

Dynamic Adaptive Decomposition

- The agent discovers subtasks at runtime
- Subtasks generated based on what the agent finds
- Better for open-ended or exploratory tasks
- Example: “Fix all bugs in this codebase” (agent must find bugs first, then fix each one)

---

9. Multi-Pass Review Pattern

Structure

1. **Per-file local passes**: each file reviewed independently for local issues
2. **Cross-file integration pass**: review relationships between files, API contracts, data flow

Why Two Passes?

- Local passes catch file-specific bugs (syntax, logic, style)
- Integration pass catches systemic issues (broken contracts, inconsistent types, circular dependencies)
- Parallel local passes, then a single integration pass

Implementation

```
Coordinator:
  → Spawn SubAgent("review", file="auth.ts")      # parallel
  → Spawn SubAgent("review", file="db.ts")         # parallel
  → Spawn SubAgent("review", file="api.ts")        # parallel
  → Wait for all results
 → Spawn SubAgent("integration-review", files=["auth.ts", "db.ts", "api.ts"])
```

---

10. Session Management

–resume

Resume a previous session to continue where you left off. Restores full conversation history.

fork_session

Create a parallel branch for exploration without affecting the main session.

update_context

Inject new information into a running session dynamically.

Scratchpad Pattern

For long-running sessions, maintain a structured scratchpad file:

```
## Key Findings
- Database schema has 3 tables: users, orders, products
- Auth uses JWT with 1-hour expiry

## Current Status
- [x] Explored database layer
```

```
- [ ] Review API routes
- [ ] Write integration tests

## Open Questions
- Why does the orders table lack a foreign key to products?
```

The agent reads/writes this file to maintain continuity across long sessions where context compression may lose details.

---

11. Workflow Patterns

Parallelization

Run multiple independent tasks simultaneously:

```
Input → [Task A, Task B, Task C] → Combine Results → Output
```

Use when subtasks are independent and can execute concurrently.

Chaining

Sequential pipeline where each step's output feeds the next:

```
Input → Step 1 → Step 2 → Step 3 → Output
```

Use when each step depends on the previous step's result.

Routing

Classify input, then direct to the appropriate handler:

```
Input → Classifier → Route A (if type X)
                   → Route B (if type Y)
                   → Route C (if type Z)
```

Use when different inputs require fundamentally different processing.

Evaluator-Optimizer

Generate output, evaluate quality, refine iteratively:

```
Input → Generator → Evaluator → (if not good enough) → Generator → ...
                                → (if good enough) → Output
```

Use when quality matters and automated evaluation is possible.

---

12. Agents vs Workflows

Dimension	Workflows	Agents
When to use	Known steps, repeatable processes	Unknown tasks, open-ended exploration
Reliability	Higher (deterministic paths)	Lower (model decides dynamically)
Flexibility	Lower (predefined structure)	Higher (adapts to novel situations)
Completion rate	Higher	Lower
Complexity	Fixed, easier to debug	Dynamic, harder to debug
Cost predictability	More predictable	Less predictable

Exam tip: If the question describes a well-defined process with known steps, the answer is likely "workflow." If the task is open-ended or requires exploration, the answer is likely "agent."

---

13. Agent Design Principles

Small Set of Abstract Tools

- Provide 4-5 well-designed tools, not 18 specialized ones
- Abstract tools are more composable
- Example: `Read`, `Write`, `Search`, `Execute` covers most needs

Environment Inspection After Actions

- After executing a tool, the agent should verify the result
- Don't assume success; check the outcome
- Example: after writing a file, read it back to confirm; after running a command, check exit code

---

14. Model Selection

Model	Best For	Trade-offs
Opus	Complex multi-step reasoning, nuanced analysis, long context tasks	Highest cost, slower
Sonnet	Balanced performance, most agentic tasks	Good cost/performance ratio
Haiku	Speed-critical tasks, simple classification, high-volume processing	Lowest cost, least capable for complex reasoning

Selection Guidelines

- Default to **Sonnet** for agentic workloads
- Use **Opus** when reasoning depth is critical (complex debugging, architectural decisions)
- Use **Haiku** for high-volume, simple tasks (classification, extraction, routing)
- Consider cost: Opus is significantly more expensive per token than Haiku

Tool Design & MCP Integration (18% of Exam)

1. Tool Descriptions: The Primary Selection Mechanism

The tool description is the single most important factor in whether a model correctly selects and uses a tool. The model reads tool descriptions to decide:

- Which tool to call
- What parameters to provide
- When to use one tool over another

What to Include in Tool Descriptions

- **Purpose**: what the tool does in one sentence
- **Input formats**: expected parameter types, acceptable values, units
- **Example queries**: concrete examples of valid invocations
- **Edge cases**: what happens with empty input, null values, boundary conditions
- **Boundary explanations**: when to use this tool vs a similar one

Example: Well-Designed Tool Description

```
{
  "name": "search_customers",
  "description": "Search for customers by name, email, or phone number. Returns
   ↪  up to 10 matching customers sorted by relevance. Use this tool when the
   ↪  user asks to find, look up, or search for a customer. Input should be a
   ↪  search query string (e.g., 'John Smith', 'john@example.com',
   ↪  '+1-555-0123'). Returns empty array if no matches found. For retrieving a
   ↪  specific customer by ID, use get_customer instead.",
  "input_schema": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "Search query: name, email, or phone number"
      }
    },
    "required": ["query"]
  }
}
```

Splitting Broad Tools into Granular Tools

Bad: One `database_query` tool that accepts arbitrary SQL

Good: Separate tools with clear purposes:

- `search_customers`: find customers by name/email/phone
- `get_customer`: retrieve a specific customer by ID
- `list_recent_orders`: get recent orders for a customer
- `update_customer_email`: change a customer's email address

Benefits of granular tools:

- Easier for the model to select the right one
- Simpler input schemas
- Better error handling per operation
- Easier to restrict access (principle of least privilege)

---

2. Structured Error Responses

Never throw exceptions or return empty strings from tools. Always return structured error objects.

Error Response Schema

```
{
  "isError": true,
  "errorCategory": "validation",
  "isRetryable": false,
  "message": "Customer ID must be a positive integer. Received: 'abc'"
}
```

Error Categories

Category	Meaning	Retryable?	Example
`transient`	Temporary failure, try again	Yes	Network timeout, rate limit
`validation`	Invalid input	No (unless input changes)	Malformed email, wrong type
`business`	Business rule violation	No	Cannot cancel shipped order

Category	Meaning	Retryable?	Example
`permission`	Access denied	No	User lacks admin role

Key Rules

- **Never throw exceptions**: the model cannot catch or interpret stack traces
- **Never return empty strings**: the model cannot distinguish “no results” from “error”
- **Distinguish access failures from valid empty results**:
  - Access failure: `{"isError": true, "errorCategory": "permission", "message": "..."}`
  - Valid empty: `{"isError": false, "results": [], "message": "No customers match query"}`

Why This Matters

The model uses error responses to decide next steps:

- `isRetryable: true` with `transient` error: model may retry
- `validation` error: model adjusts input and tries again
- `permission` error: model escalates or informs the user
- `business` error: model explains the constraint to the user

---

3. Scoped Tool Access

Principle: 4-5 Tools Per Agent

- Each agent should have access to a small, focused set of tools
- Do not give a single agent 18 tools
- Reduces confusion, improves selection accuracy
- Aligns with principle of least privilege

Example Scoping

```
Coordinator Agent:
  tools: [Task]  # Can only spawn subagents

Research Agent:
```

```
  tools: [Grep, Glob, Read, web_search]  # Read-only exploration

Editor Agent:
  tools: [Read, Write, Edit, Bash]  # Can modify files

Reviewer Agent:
  tools: [Read, Grep, Glob]  # Read-only analysis
```

---

4. tool_choice Parameter

Controls how the model uses tools on each turn.

Value	Behavior	Use Case
`"auto"`	Model may or may not call a tool	Default, most flexible
`"any"`	Model must call at least one tool	Force tool use when you know a tool is needed
`{"type": "tool", "name": "..."}`	Model must call this specific tool	Force a particular tool call

Code Example

```python
# Auto: model decides
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    tools=tools,
    tool_choice={"type": "auto"},
    messages=messages,
)

# Any: must use a tool
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    tools=tools,
    tool_choice={"type": "any"},
    messages=messages,
)

# Forced: must use specific tool
```

```
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    tools=tools,
    tool_choice={"type": "tool", "name": "search_customers"},
    messages=messages,
)
```

---

5. MCP (Model Context Protocol)

Server Scoping

File	Scope	Version Controlled?	Use Case
`.mcp.json`	Project-level	Yes	Shared project tools
`~/.claude.json`	Personal/global	No	Personal tools, API keys

Environment Variable Expansion

`.mcp.json` supports environment variable expansion for secrets:

```
{
  "mcpServers": {
    "database": {
      "command": "mcp-server-postgres",
      "args": ["--connection-string", "${DATABASE_URL}"]
    }
  }
}
```

This keeps secrets out of version control while allowing project-level MCP configuration.

MCP Primitives

Primitive	Controlled By	Description
Resources	Application (app-controlled)	Content catalogs the application exposes to the model. The app decides when to attach resources to context.
Tools	Model (model-controlled)	Functions the model can call. The model decides when and how to invoke tools.
Prompts	User (user-controlled)	Pre-built prompt templates the user selects. The user triggers prompt execution.

MCP Transports

Stdio Transport

- **Same-machine** communication
- Full bidirectional streaming
- Uses stdin/stdout pipes
- Best for local tools and development
- Supports all MCP features (progress, sampling, etc.)

```
{
  "mcpServers": {
    "local-db": {
      "command": "mcp-server-sqlite",
      "args": ["/path/to/database.db"]
    }
  }
}
```

StreamableHTTP Transport

- **Remote** communication over HTTP
- Server-to-client communication uses SSE (Server-Sent Events)
- Limited compared to Stdio (some features not available)
- Best for remote/hosted MCP servers

Stateless HTTP Flag

- Disables sessions, progress reporting, and sampling

- Use when the server is a simple request-response API
- Reduces complexity for simple tool servers

MCP Sampling

MCP Sampling allows **servers to request LLM generation from clients**. This inverts the normal flow:

Normal: Client sends prompt to server's LLM Sampling: Server asks client's LLM to generate text

Use case: an MCP server needs the model to analyze or transform data as part of tool execution.

---

6. Built-in Tools in Claude Code

Tool	Purpose	Notes
`Grep`	Content search (ripgrep-based)	Search file contents by regex
`Glob`	File pattern matching	Find files by name patterns
`Read`	Read file contents	Supports line ranges
`Write`	Create/overwrite files	Must Read first for existing files
`Edit`	Surgical text replacement	Preferred over Write for modifications
`Bash`	Shell command execution	Full shell access

Edit Fallback Pattern

When `Edit` fails because `old_string` is not unique in the file:

1. Use `Read` to get the full file contents
2. Use `Write` to overwrite with the corrected version

This is the standard fallback when Edit cannot find a unique match.

---

7. Batch Tool Pattern

Multiple tools can be called in a single model response turn. This enables parallel execution:

```
// Model response includes multiple tool_use blocks:
[
  {"type": "tool_use", "id": "1", "name": "read_file", "input": {"path":
   ↪  "a.ts"}},
  {"type": "tool_use", "id": "2", "name": "read_file", "input": {"path":
   ↪  "b.ts"}},
  {"type": "tool_use", "id": "3", "name": "read_file", "input": {"path":
   ↪  "c.ts"}}
]
```

All three reads execute in parallel, and all results are returned together. This reduces round trips and speeds up agentic execution.

---

8. Built-in API Tools

Web Search Tool

Claude API includes a built-in `web_search` tool:

```
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    tools=[{"type": "web_search_20250305"}],
    messages=[{"role": "user", "content": "What is the current price of AAPL?"}],
)
```

- No MCP server needed
- Built into the API
- Model decides when to search

Text Editor Tool

Built-in tool for file editing in the API (separate from Claude Code's Edit tool):

```
tools=[{
    "type": "text_editor_20250429",
    "name": "str_replace_based_edit_tool"
}]
```

Computer Use

- A tool system for controlling a computer (mouse, keyboard, screen)
- Typically runs inside a Docker container for safety
- The model sees screenshots and issues actions
- Use for tasks requiring GUI interaction

---

9. Tool Design Principles Summary

1. **Descriptions are everything**: the model selects tools based on descriptions alone
2. **One tool, one purpose**: split broad tools into focused, single-purpose tools
3. **Structured errors**: always return `isError`, `errorCategory`, `isRetryable`
4. **Never throw, never return empty**: the model needs interpretable responses
5. **Scope access**: 4-5 tools per agent, not the entire toolkit
6. **Distinguish empty from error**: valid empty results are not errors
7. **Include examples in descriptions**: concrete usage examples improve selection
8. **Edge cases in descriptions**: document boundary conditions and limitations

Claude Code Configuration & Workflows (20% of Exam)

1. CLAUDE.md Hierarchy

CLAUDE.md files provide persistent instructions that Claude Code loads automatically. They form a hierarchy with increasing specificity.

Levels

Level	Location	Shared via VCS?	Scope
User-level	`~/.claude/CLAUDE.md`	No (personal)	All projects for this user
Project-level	`.claude/CLAUDE.md` or `CLAUDE.md` at repo root	Yes	Everyone working on this project
Directory-level	`CLAUDE.md` in any subdirectory	Yes	Active when working in that directory

Key Points

- **User-level is NOT shared** via version control. It is personal to the developer.
- Project-level is checked into the repo and shared with the team.
- Directory-level provides context-specific instructions for subdirectories.
- All levels are merged; more specific levels can override broader ones.

@import Syntax

CLAUDE.md files support `@import` for modular organization:

```
# Project Instructions

@import ./coding-standards.md
@import ./api-conventions.md
@import ./testing-requirements.md
```

This keeps CLAUDE.md files manageable by splitting instructions into focused modules.

---

2. .claude/rules/ Directory

Rules files support YAML frontmatter with glob-pattern matching for conditional loading.

Structure

```
.claude/
  rules/
    typescript.md      # Loaded for .ts files
    testing.md         # Loaded for test files
    api-routes.md      # Loaded for API route files
```

YAML Frontmatter with Glob Patterns

```yaml
---
paths:
  - "src/**/*.ts"
  - "src/**/*.tsx"
---

# TypeScript Rules

- Always use explicit return types on exported functions
- Prefer interfaces over type aliases for object shapes
```

```yaml
---
paths:
  - "**/*.test.ts"
  - "**/*.spec.ts"
---

# Testing Rules

- Use describe/it blocks, not test()
- Always include edge case tests
```

Rules are only loaded when the agent is working on files matching the glob patterns.

---

3. Custom Slash Commands

Project Commands

Location: `.claude/commands/`

Shared with the team via version control.

```
<!-- .claude/commands/review.md -->
Review the current changes for:
1. Type safety issues
2. Missing error handling
3. Security vulnerabilities

Run `tsc --noEmit` to verify types compile.
```

Invoked as: `/project:review`

Personal Commands

Location: `~/.claude/commands/`

Personal, not shared via version control.

```
<!-- ~/.claude/commands/ship.md -->
Commit all changes and push to the remote branch.
```

Invoked as: `/user:ship`

---

4. Skills: SKILL.md

Skills are reusable capabilities defined with a `SKILL.md` file and frontmatter.

Frontmatter Options

```
---
context: fork
allowed-tools:
  - Read
  - Grep
  - Glob
```

```
argument-hint: "path to file or directory to analyze"
---

# Code Analysis Skill

Analyze the provided code for common issues...
```

Field	Purpose
`context: fork`	Run in an isolated forked session (prevents context pollution)
`allowed-tools`	Restrict which tools this skill can use
`argument-hint`	Help text shown to the user for the skill's argument

---

5. Plan Mode vs Direct Execution

Plan Mode

- For **complex architectural tasks** requiring thought before action
- Claude analyzes the problem, proposes a plan, then executes
- Breadth-first exploration: considers multiple approaches
- Use when the scope is large or unclear

Direct Execution

- For **simple, well-scoped changes** where the path is clear
- Claude executes immediately without a planning phase
- Faster for straightforward tasks
- Use when you know exactly what needs to change

Plan Mode vs Thinking Mode

Mode	Focus	When to Use
Plan Mode	Breadth: explore multiple approaches	Complex multi-file changes, architecture decisions

Mode	Focus	When to Use
Thinking Mode	Depth: reason deeply about one approach	Tricky logic, debugging, nuanced decisions

---

6. Key Interaction Patterns

Explore Subagent

Spawn a subagent specifically for codebase exploration. This isolates verbose discovery output (file listings, search results) from the main conversation, keeping the coordinator's context clean.

Interview Pattern

Claude asks clarifying questions before implementing. Useful for ambiguous requirements:

```
User: "Add authentication to the app"
Claude: "Before I implement authentication, I have a few questions:
  1. Which auth provider (OAuth, JWT, session-based)?
  2. Which routes need protection?
  3. Do you need role-based access control?"
```

Iterative Refinement

- Provide **concrete input/output examples** for the desired behavior
- Use **test-driven iteration**: write tests first, then implement until tests pass
- Each iteration narrows the gap between current and desired behavior

---

7. CI/CD Integration

Non-Interactive Mode

Use –p flag for non-interactive (headless) execution:

```
claude -p "Run the test suite and report results"
```

Structured Output

Use `--output-format json` with `--json-schema` for machine-readable output:

```
claude -p "Analyze code quality" \
  --output-format json \
  --json-schema '{"type": "object", "properties": {"score": {"type": "number"},
  ↪  "issues": {"type": "array"}}}'
```

This enables integration with CI/CD pipelines, dashboards, and automated workflows.

---

8. Session Context Isolation

Self-Review Limitation

The session that generated code is less effective at reviewing it because it retains the reasoning context. The model "remembers" its intent and is biased toward seeing it as correct.

Solution: use a separate session (or separate subagent) for review.

Providing Existing Test Files

When working with existing test files, provide them to Claude to prevent duplicate test suggestions. Without seeing existing tests, Claude may suggest tests that already exist.

---

9. Claude Code SDK

Default Behavior

The SDK is **read-only by default**. Write operations must be explicitly enabled:

```
import { createAgent } from "@anthropic-ai/claude-code-sdk";

const agent = createAgent({
  allowTools: ["Read", "Grep", "Glob", "Write", "Edit"],
  // Without Write and Edit, the agent can only read
});
```

Key Configuration

- `allowTools`: explicitly list which tools the agent can use
- Default is read-only (safe for analysis tasks)
- Must opt-in to write capabilities

---

10. GitHub Integration

- **`/install`**: installs the GitHub app for the repository
- **Mention support**: @mention Claude in PR comments for review
- **PR review**: Claude can review pull requests, suggest changes
- **Git worktrees**: use for running parallel Claude instances on different branches without conflicts

```
# Create worktrees for parallel work
git worktree add ../feature-a feature-a
git worktree add ../feature-b feature-b
# Run separate Claude instances in each worktree
```

---

11. /init Command

The `/init` command creates a `CLAUDE.md` file for the project. It analyzes the codebase and generates initial instructions covering:

- Project structure
- Build commands
- Testing conventions
- Code style preferences

---

12. Three Memory Types

Type	Location	Shared?	Purpose
Project memory	`.claude/CLAUDE.md`	Yes (VCS)	Team-wide project instructions

Type	Location	Shared?	Purpose
Local memory	`.claude/local/CLAU` (gitignored)	No	Personal project overrides
User memory	`~/.claude/CLAUDE.md`	No	Global personal preferences

---

13. Keyboard Controls

Action	Effect
Escape	Stop current generation
Double Escape	Rewind to before last action
Compact	Summarize conversation to reduce context
Clear	Fresh start, new session

Compact vs Clear

- **Compact**: summarizes the conversation but maintains continuity. Use when context is getting long but you want to keep working on the same task.
- **Clear**: completely fresh start. Use when switching to a new task.

---

14. Hooks System

Hooks are scripts that run before or after tool executions.

PreToolUse Hooks

Run **before** a tool executes. Can allow or block the tool call.

Exit Code	Meaning
`0`	Allow the tool call to proceed
2	Block the tool call

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write",
        "command": "python check_write_path.py $TOOL_INPUT"
      }
    ]
  }
}
```

PostToolUse Hooks

Run **after** a tool executes. Cannot block (the tool already ran).

Used for:

- Logging and auditing
- Data normalization
- Triggering follow-up actions

Hook Pattern: TypeScript Type Checker

Run `tsc --noEmit` after every file edit to catch type errors immediately:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "command": "npx tsc --noEmit 2>&1 | head -20"
      }
    ]
  }
}
```

Hook Pattern: Duplicate Code Prevention

A PreToolUse hook that checks for duplicate code before allowing writes:

```
{
  "hooks": {
    "PreToolUse": [
      {
```

```
        "matcher": "Write",
        "command": "python check_duplicates.py $TOOL_INPUT"
      }
    ]
  }
}
```

---

15. Configuration Quick Reference

Configuration	File	Scope
Project instructions	`.claude/CLAUDE.md`	Project (shared)
Local overrides	`.claude/local/CLAUDE.md`	Personal (gitignored)
User preferences	`~/.claude/CLAUDE.md`	Global (personal)
Rules with globs	`.claude/rules/*.md`	Conditional per file pattern
Project commands	`.claude/commands/*.md`	Project (shared)
Personal commands	`~/.claude/commands/*.md`	Global (personal)
MCP servers (project)	`.mcp.json`	Project (shared)
MCP servers (personal)	`~/.claude.json`	Global (personal)
Hooks and settings	`.claude/settings.json`	Project (shared)
Skills	`SKILL.md`	Per skill directory

Prompt Engineering & Structured Output (20% of Exam)

1. Explicit Criteria Over Vague Instructions

Bad: Vague

> "Check that comments are accurate"

This is ambiguous. What counts as "accurate"? The model will apply inconsistent judgment.

Good: Explicit

> "Flag any comment where the described behavior contradicts the actual code logic. Specifically check: return values, error conditions, parameter descriptions, and side effects."

Why This Matters

- Explicit criteria produce consistent results across invocations
- Vague instructions lead to high false positive rates
- **High false positive rates undermine trust** in the accurate categories (if the model cries wolf too often, users ignore all flags)

---

2. Few-Shot Examples

Few-shot examples are concrete input-output pairs included in the prompt to guide the model's behavior.

Purposes

1. **Consistent formatting**: ensure output matches a specific structure
2. **Ambiguous case handling**: show the model how to handle edge cases
3. **Generalization**: demonstrate the pattern so the model applies it to novel inputs

Reducing Hallucination

Include examples that show correct handling of null/missing data:

```
<example>
  <input>Customer: John Smith, Email: null, Phone: +1-555-0123</input>
  <output>
    {
      "name": "John Smith",
      "email": null,
      "phone": "+15550123",
      "status": "partial_record"
    }
  </output>
</example>

<example>
  <input>Customer: , Email: , Phone: </input>
  <output>
    {
      "name": null,
      "email": null,
      "phone": null,
      "status": "empty_record"
    }
  </output>
</example>
```

By showing explicit null-handling examples, you prevent the model from inventing data to fill gaps.

Standardized Formats

Few-shot examples enforce standardized output formats more reliably than instructions alone:

```
<example>
  <input>Date: March 15th, 2024</input>
  <output>2024-03-15</output>
</example>

<example>
  <input>Date: 15/03/2024</input>
  <output>2024-03-15</output>
</example>
```

---

3. tool_use with JSON Schemas

What Schemas Solve

- **Eliminates syntax errors**: output is guaranteed valid JSON matching the schema
- Does **NOT** eliminate semantic errors: the model may still put wrong values in correct fields

Schema Example

```
tools = [{
    "name": "classify_ticket",
    "description": "Classify a support ticket by category and priority",
    "input_schema": {
        "type": "object",
        "properties": {
            "category": {
                "type": "string",
                "enum": ["billing", "technical", "account", "other"]
            },
            "priority": {
                "type": "string",
                "enum": ["low", "medium", "high", "critical"]
            },
            "summary": {
                "type": "string",
                "description": "One-sentence summary of the issue"
            }
        },
        "required": ["category", "priority", "summary"]
    }
}]
```

tool_choice Options (Recap)

Value	Model Behavior
`"auto"`	May or may not use a tool
`"any"`	Must call at least one tool
`{"type": "tool", "name": "..."}`	Must call the specified tool

4. Schema Design Patterns

Optional (Nullable) Fields

Use nullable fields for data that may not be present:

```
{
  "type": "object",
  "properties": {
    "name": {"type": "string"},
    "email": {"type": ["string", "null"]},
    "phone": {"type": ["string", "null"]}
  },
  "required": ["name", "email", "phone"]
}
```

All fields are required in the response, but `email` and `phone` can be `null`. This forces the model to explicitly acknowledge missing data instead of hallucinating values.

Resilient Catch-All Pattern (Extensible Enums)

Good: Include "other" with a detail field:

```
{
  "category": {
    "type": "string",
    "enum": ["billing", "technical", "account", "other"]
  },
  "category_detail": {
    "type": ["string", "null"],
    "description": "If category is 'other', provide a brief description"
  }
}
```

Bad (Fragile Expansion Anti-Pattern): Adding every possible value:

```
{
  "category": {
    "type": "string",
    "enum": ["billing", "technical", "account", "shipping", "returns",
     ↪  "warranty", "loyalty", "gift_cards", ...]
  }
}
```

The fragile pattern breaks every time a new category appears. The catch-all pattern gracefully handles novel inputs.

Schema Redundancy for Validation

Include calculated fields alongside stated fields for cross-validation:

```
{
  "stated_total": {
    "type": "number",
    "description": "The total as stated in the document"
  },
  "calculated_total": {
    "type": "number",
    "description": "The total calculated by summing line items"
  },
  "totals_match": {
    "type": "boolean",
    "description": "Whether stated_total equals calculated_total"
  }
}
```

This enables downstream validation: if `totals_match` is false, the extraction may have errors.

detected_pattern Field

Include a field for the model to explain its reasoning for systematic analysis:

```
{
  "classification": "false_positive",
  "detected_pattern": "Comment references legacy behavior that was true before
  ↪  refactor in commit abc123",
  "confidence": 0.85
}
```

The `detected_pattern` field enables systematic false positive analysis across runs.

---

5. Retry-with-Error-Feedback

When It Works

Effective for **format errors**: the model produced output in the wrong format, and the error message tells it exactly what was wrong.

```
Error: Expected ISO-8601 date format (YYYY-MM-DD), got "March 15, 2024"
Please retry with the correct format.
```

When It Fails

Ineffective for **missing information**: if the source data does not contain the needed information, no amount of retrying will produce it.

```
Error: "customer_email" field is null
Please retry and provide the email.
```

If the email was not in the source document, retrying just wastes tokens or, worse, causes the model to hallucinate a value.

---

6. Message Batches API

Key Properties

Property	Value
Cost savings	50% discount compared to standard API
Processing window	24 hours (results available within this time)
Multi-turn tool calling	Not supported (single-turn only)
Identification	Each request needs a `custom_id` for matching results

When to Use

- Large-scale batch processing (thousands of documents)
- Non-time-sensitive tasks
- Cost optimization is a priority
- Tasks that do not require multi-turn interaction

Batch Frequency Calculation

Given an SLA constraint (e.g., “results within 24 hours”), calculate how frequently to submit batches:

- If new data arrives continuously: submit batches at intervals that ensure all items are processed within 24 hours
- If data arrives in bursts: batch the entire burst and submit once

---

7. Review Patterns

Self-Review Limitations

The session that generated the output retains its reasoning context. This creates bias:

- The model "knows" what it intended, so it reads correctness into ambiguous output
- It is less likely to catch its own systematic errors
- The retained context makes the model overconfident

Multi-Pass Review

Per-file local passes: review each file independently for file-specific issues (syntax, logic, style, security).

Cross-file integration pass: review relationships between files for systemic issues (API contract mismatches, inconsistent types, data flow problems).

Independent Review Instances

Use a **separate session or subagent** for review instead of self-review. The independent reviewer has no prior context about the intent, so it evaluates the output purely on its merits.

Confidence-Based Routing

Route items to human review based on model confidence:

```
{
  "result": "approved",
  "confidence": 0.62,
  "route_to_human": true,
  "reason": "Confidence below 0.80 threshold"
}
```

- High confidence (>0.95): auto-approve
- Medium confidence (0.80-0.95): spot-check
- Low confidence (<0.80): mandatory human review

---

8. Temperature

Value	Behavior	Use Case
`0`	Deterministic (most likely tokens)	Classification, extraction, code generation
`0.5`	Balanced	General-purpose tasks
`1`	Creative (more random sampling)	Creative writing, brainstorming

Exam tip: For structured output and classification tasks, use temperature 0. For creative tasks, use higher temperatures.

---

9. Response Prefilling + Stop Sequences

Prefilling

Pre-populate the assistant's response to force a specific format:

```
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    messages=[
        {"role": "user", "content": "Extract the date from: 'Meeting on March
↪   15'"},
        {"role": "assistant", "content": "{\"date\": \""}
    ],
    stop_sequences=["\"}"]
)
# Response will be just: 2024-03-15
```

Stop Sequences

Stop generation when a specific string is encountered. Useful for:

- Extracting specific values without trailing text
- Forcing concise responses
- Delimiting structured output

---

10. Extended Thinking

How It Works

- Separate reasoning phase before the main response
- The model "thinks" step by step in a dedicated thinking block
- Minimum **1024 tokens** for the thinking budget
- Thinking content is visible in the API response but can be hidden from users

When to Use

- **Complex multi-step reasoning** where intermediate steps matter
- **Math and logic** problems requiring explicit calculation
- **Ambiguous inputs** where the model needs to consider multiple interpretations

Decision Framework

Use **eval-driven decision making**: run the task with and without extended thinking, measure quality, use thinking only when it measurably improves results.

---

11. System Prompts

Purpose

- **Role assignment**: "You are a senior code reviewer specializing in security"
- **Behavior control**: "Always respond in JSON format"
- **Context setting**: "You are reviewing a Python 3.11 codebase using FastAPI"

Best Practices

- Be clear and direct: use action verbs ("Flag", "Extract", "Classify")
- First line matters most: lead with the most important instruction
- Be specific about what to do AND what not to do

---

12. Being Specific: Type A and Type B Guidelines

Type A: Attribute Guidelines

Define the **qualities** of a good output:

- "The summary should be under 100 words"
- "Use active voice"
- "Include specific numbers, not vague qualifiers"

Type B: Step-by-Step Guidelines

Define the **process** to follow:

1. "First, identify all function signatures"
2. "Then, check each function for missing type annotations"
3. "Finally, report functions with missing annotations"

Best practice: combine both types. Type A defines what "good" looks like; Type B defines how to get there.

---

13. XML Tags for Content Structure

XML tags organize complex prompts clearly:

```
<instructions>
Classify the following customer message into a category.
</instructions>

<categories>
- billing: Payment, charges, refunds, invoices
- technical: Bugs, errors, feature requests, integrations
- account: Login, permissions, profile, settings
</categories>

<message>
{{customer_message}}
</message>

<output_format>
Respond with a JSON object: {"category": "...", "confidence": 0.0-1.0}
</output_format>
```

Benefits:

- Clear visual separation of prompt components
- Easy to reference specific sections
- Model understands the structure unambiguously

---

14. Prompt Evaluation Workflow

The Cycle

1. **Draft**: write the initial prompt
2. **Dataset**: assemble test inputs with known-correct outputs
3. **Execute**: run the prompt against all test inputs
4. **Grade**: evaluate outputs against expected results
5. **Iterate**: refine the prompt based on grading results

Grading Methods

Code-based grading: programmatic validation

- Syntax validation (is the JSON valid?)
- Schema conformance (does it match the expected structure?)
- Exact match on specific fields
- Regex patterns for format checking

Model-based grading: LLM evaluates quality

- Semantic correctness (is the answer factually right?)
- Tone and style assessment
- Completeness evaluation
- Comparison against reference answers

Best practice: use both. Code-based for format and structure; model-based for content quality.

---

15. AI Fluency: The 4 Ds

Delegation

Knowing which tasks to assign to AI and how to scope them effectively.

- Break large tasks into smaller, well-defined subtasks
- Provide clear success criteria
- Know when a task is too ambiguous for AI

Description

Ability to clearly describe what you want in natural language.

- Explicit criteria over vague instructions
- Concrete examples over abstract descriptions
- Specify both what to do and what to avoid

Discernment

Ability to evaluate AI output quality and catch errors.

- Recognize hallucinations
- Spot logical errors in generated code
- Identify when the model is confidently wrong

Diligence

Systematic follow-through on verification and iteration.

- Always verify AI output before using it
- Iterate on prompts when results are not satisfactory
- Maintain testing and evaluation practices

---

16. Human-AI Interaction Modes

Mode	Human Role	AI Role	Example
Automation	Sets up, then hands off	Executes independently	Batch document processing
Augmentation	Drives the process	Assists and enhances	Code completion, suggestion
Agency	Sets goals	Plans and executes with autonomy	Agentic coding, autonomous research

Key Differences

- **Automation**: AI runs without human interaction after setup. Human trusts the pipeline.

- **Augmentation**: human is always in the loop, AI provides suggestions. Human makes final decisions.
- **Agency**: AI has significant autonomy but operates within guardrails. Human sets boundaries and reviews outcomes.

Context Management & Reliability (15% of Exam)

1. Tool Context Pruning

When tools return verbose output, prune it on the application side before passing it back to the model.

Why Prune

- Verbose tool responses consume context window space
- Irrelevant details distract the model from the actual task
- Cost increases with context size

How to Prune

```
def prune_tool_response(raw_output, relevant_fields):
    """Filter tool output to only include relevant information."""
    parsed = json.loads(raw_output)
    return {k: v for k, v in parsed.items() if k in relevant_fields}

# Example: database query returns 50 columns, but we only need 3
raw = tool.execute("SELECT * FROM orders WHERE id = 123")
pruned = prune_tool_response(raw, ["order_id", "status", "total"])
```

Application-Side Filtering

The application (not the model) decides what to filter. This is done in the tool execution layer, before results are sent back to the model as `tool_result` messages.

---

2. Long Session Compression

Strategy

- **Summarize resolved turns**: conversations about issues that have been fixed can be compressed to a brief summary
- **Keep verbatim for active issues**: anything the model is still working on should remain in full detail

Example

```
# Before compression (20 turns about 3 issues):

Turn 1-5: Debugging auth bug (RESOLVED)
Turn 6-10: Refactoring database layer (RESOLVED)
Turn 11-20: Implementing new API endpoint (IN PROGRESS)

# After compression:

[Summary] Fixed auth bug by updating JWT validation in auth.ts.
[Summary] Refactored database layer to use connection pooling.
Turn 11-20: [Full detail preserved for active work]
```

Compact Command

In Claude Code, the Compact command triggers conversation summarization. It reduces context length while preserving key information about the current task.

---

3. Resuming Async Sessions

When resuming a session after an interruption:

1. **Filter out `tool_result` messages**: stale tool results may no longer be accurate
2. **Force re-fetch**: have the model re-execute tools to get current data
3. **Provide a summary** of where things left off

Why Filter tool_result Messages

- File contents may have changed since the tool was run
- Database state may have evolved
- External APIs may return different results
- Stale data leads to incorrect decisions

---

4. Scratchpad Pattern

For extended sessions, maintain a structured file that the agent reads and writes to preserve key findings.

Structure

```
# Session Scratchpad

## Architecture Understanding
- Monorepo with 3 packages: api, web, shared
- Database: PostgreSQL with Prisma ORM
- Auth: JWT-based, tokens stored in httpOnly cookies

## Key Findings
- Bug in orders/route.ts: missing null check on line 47
- Performance issue: N+1 query in getOrdersWithItems()
- Security: API key exposed in client-side bundle

## Current Task
Implementing fix for the N+1 query problem

## Completed
- [x] Mapped the full codebase structure
- [x] Identified all critical bugs
- [x] Fixed auth token expiry bug

## Next Steps
- [ ] Fix N+1 query with eager loading
- [ ] Add input validation to POST /orders
- [ ] Write integration tests
```

Why Use a Scratchpad

- Context compression may lose important details
- The scratchpad persists on disk, surviving any context manipulation
- The agent can re-read the scratchpad after a Compact or resume
- Acts as external memory for the agent

---

5. Directed Codebase Exploration

Strategy

1. **Start broad**: use Glob and Grep to understand project structure
2. **Dynamically generate subtasks**: based on what the exploration reveals
3. **Narrow progressively**: drill into specific files and functions

Example Flow

```
Step 1: Glob("**/*.ts") → understand file layout
Step 2: Read("package.json") → understand dependencies
Step 3: Grep("export.*function") → find public API surface
Step 4: Read specific files based on findings
Step 5: Generate targeted analysis tasks
```

This is better than randomly reading files because each step informs the next.

---

6. Graceful Tool Failure

Rules

1. **Use `isError` flag**: always return a structured error response
2. **Never throw exceptions**: the model cannot interpret stack traces
3. **Never return empty strings**: indistinguishable from “no data”

Error Response Structure

```json
{
  "isError": true,
  "errorCategory": "transient",
  "isRetryable": true,
  "message": "Database connection timed out after 5000ms"
}
```

Success with Empty Results

```json
{
  "isError": false,
  "results": [],
  "message": "No customers found matching query 'xyz123'"
}
```

The model can distinguish “the search worked but found nothing” from “the search failed.”

---

7. Human-in-the-Loop Calibration

Field-Level Confidence

Instead of a single confidence score per item, provide confidence per field:

```
{
  "name": {"value": "John Smith", "confidence": 0.99},
  "email": {"value": "john@example.com", "confidence": 0.95},
  "phone": {"value": "+1-555-0123", "confidence": 0.72},
  "address": {"value": "123 Main St", "confidence": 0.45}
}
```

Validate Across Segments

Do not just check aggregate accuracy. Validate across different segments:

- Different input types (short vs long documents)
- Different languages or formats
- Edge cases (missing fields, unusual formats)

A model might have 95% aggregate accuracy but only 60% accuracy on a specific segment.

---

8. Escalation Patterns

Honor Immediate Requests

If a customer says “I want to speak to a manager,” escalate immediately. Do not try to resolve first.

Gather Context for Complex Issues

For complex escalations, gather context before handing off:

1. Identify the customer (ID, account details)
2. Understand the root cause
3. Document what has been attempted
4. Prepare a structured summary

Structured Summary Payload

```
{
  "customer_id": "C-12345",
  "root_cause": "Duplicate charge on order #98765",
  "amount": 49.99,
  "currency": "USD",
  "recommended_action": "Full refund",
  "attempted_resolutions": [
    "Verified order status: shipped",
    "Confirmed duplicate charge in payment system"
  ],
  "conversation_summary": "Customer noticed two $49.99 charges..."
}
```

---

9. API Statelessness

Core Principle

The Claude API is **stateless**. The server does not remember previous requests. Every API call must include the **entire conversation history**.

Implementation

```
# You maintain the conversation history
messages = []

# Turn 1
messages.append({"role": "user", "content": "Hello"})
response = client.messages.create(messages=messages, ...)
messages.append({"role": "assistant", "content": response.content})

# Turn 2: must include ALL previous messages
messages.append({"role": "user", "content": "What did I just say?"})
response = client.messages.create(messages=messages, ...)  # includes turn 1
```

Multi-Turn Conversation Management

- Maintain a list of messages on the client side
- Append each user message and assistant response
- Send the full list with every API call
- Prune old messages when approaching context limits

---

10. Prompt Caching

What It Does

Prompt caching reuses computational work from previous requests that share the same prefix. Instead of re-processing the entire prompt, the API recognizes cached portions and skips redundant computation.

TTL (Time to Live)

Platform	Cache TTL
Anthropic API	1 hour
Amazon Bedrock	5 minutes

Cache Breakpoints

- **Minimum 1024 tokens** for a cache breakpoint to be effective
- Up to **4 breakpoints** per request
- Cache is **invalidated by content changes**: if any token before the breakpoint changes, the cache is invalid

Processing Order

When the API processes a request with tools, the order is:

1. **Tools** (tool definitions)
2. **System prompt**
3. **Messages** (conversation history)

This order matters for caching: tools and system prompt are processed first, so they benefit most from caching (they tend to be stable across requests).

Caching Strategy

Place stable content early in the request:

```
[Tool definitions]       ← Rarely change, great cache candidates
[System prompt]          ← Usually stable
[Early conversation]     ← Stable once added
[Recent messages]        ← Change every turn, not cached
```

---

11. Response Streaming

SSE (Server-Sent Events)

The API streams responses as SSE events:

```
event: content_block_start
data: {"type": "content_block_start", "index": 0, ...}

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "text_delta", "text

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "text_delta", "text

event: message_stop
data: {"type": "message_stop"}
```

Key Event Types

Event	Purpose
`message_start`	Response metadata
`content_block_start`	New content block begins
`content_block_delta`	Incremental text chunk
`content_block_stop`	Content block complete
`message_stop`	Full response complete

content_block_delta

This is the event that delivers actual text content. Each delta contains a small text chunk. Concatenating all deltas in order produces the full response.

---

12. Citations

Types

Citation Type	Source Format	Location Format
Page locations	PDF documents	Page number + bounding box
Char locations	Plain text	Start and end character offsets

Purpose

Citations enable **source verification**: the user can trace any claim back to the exact location in the source document.

Usage

```
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    messages=[{
        "role": "user",
        "content": [
            {"type": "document", "source": {"type": "base64", ...}},
           {"type": "text", "text": "Summarize the key findings with citations"}
        ]
    }]
)
# Response includes citation objects pointing to specific locations
```

---

13. Context Window

The context window is the **total information capacity** of a single API call. It includes:

- System prompt
- Tool definitions
- All messages (user, assistant, tool_result)
- The model's response

Varies by Model

Different models have different context window sizes. Larger windows allow more conversation history but cost more.

Managing Context

When approaching the limit:

1. Summarize old turns (compression)
2. Remove resolved conversation threads
3. Use the scratchpad pattern for persistent memory
4. Start a new session for unrelated tasks

---

14. RAG Pipeline (Retrieval-Augmented Generation)

Full Pipeline

`Query → Embedding → Vector Search → Retrieved Chunks → Prompt Assembly → LLM → Response`

Steps

1. **Chunking**: split documents into smaller pieces
2. **Embedding**: convert chunks to numerical vectors
3. **Vector storage**: store embeddings in a vector database
4. **Similarity search**: find chunks most similar to the query
5. **Prompt assembly**: combine retrieved chunks with the user's question
6. **LLM generation**: the model answers using the retrieved context

---

15. Chunking Strategies

Size-Based Chunking

Split by fixed token/character count with overlap:

```
Document: [.........................................]
Chunk 1:  [===========]
Chunk 2:       [===========]     (overlap)
Chunk 3:            [===========]
```

- **Overlap** ensures that information at chunk boundaries is not lost
- Simple to implement
- May split mid-sentence or mid-paragraph

Structure-Based Chunking

Split by document structure (headings, paragraphs, sections):

```
# Section 1        → Chunk 1
Paragraph text...

# Section 2        → Chunk 2
Paragraph text...

## Subsection 2.1  → Chunk 3
Paragraph text...
```

- Preserves semantic coherence within chunks
- Chunk sizes vary
- Requires document structure parsing

Semantic-Based Chunking

Split by meaning, grouping related content:

- Uses embeddings to detect topic boundaries
- Groups semantically similar sentences together
- Most sophisticated, best retrieval quality
- Most expensive to compute

---

16. Embeddings

What They Are

Numerical vectors (arrays of floating-point numbers) that represent the semantic meaning of text. Texts with similar meaning have similar vectors.

Example

```
"The cat sat on the mat"  → [0.23, -0.41, 0.87, ...]  (1536 dimensions)
"A feline rested on a rug" → [0.21, -0.39, 0.85, ...]  (similar vector)
"The stock market crashed" → [-0.62, 0.13, -0.44, ...] (very different vector)
```

Cosine Similarity / Distance

Measures how similar two vectors are:

- **Cosine similarity**: 1.0 = identical, 0.0 = unrelated, -1.0 = opposite
- **Cosine distance**: 1 - cosine_similarity (0.0 = identical, 2.0 = opposite)

Used for vector search: find chunks whose embedding vectors are most similar to the query embedding.

---

17. Hybrid Search

Combines vector search with traditional lexical search for better retrieval.

Components

Method	Strength	Weakness
Vector search	Semantic similarity (understands meaning)	Misses exact keyword matches
BM25 lexical search	Exact keyword matching	No semantic understanding

Reciprocal Rank Fusion (RRF)

Merges results from both search methods:

```
Score(doc) = Σ 1 / (k + rank_i(doc))
```

Where `rank_i(doc)` is the document's rank in search method `i`, and k is a constant (typically 60).

Documents that rank highly in both methods get the highest combined score.

---

18. Reranking

After initial retrieval, use an LLM to reorder results by relevance.

Pipeline with Reranking

```
Query → Initial Retrieval (fast, broad) → Top 50 candidates
      → Reranker (LLM, slow, precise) → Top 5 most relevant
      → Prompt Assembly → LLM → Response
```

Why Rerank

- Initial retrieval is fast but approximate
- The reranker considers the full context of query + document
- Dramatically improves precision of the final results
- Worth the extra cost for quality-critical applications

---

19. Contextual Retrieval

Problem

Standard chunks lose document context. A chunk saying “Revenue increased 15%” is meaningless without knowing which company or time period.

Solution

Add document-level context to each chunk **before** embedding:

```
Original chunk:
"Revenue increased 15% year-over-year."

Contextualized chunk:
"[From Acme Corp 2024 Q3 Earnings Report, Section: Financial Highlights]
Revenue increased 15% year-over-year."
```

Benefits

- Each chunk is self-contained with its context
- Embeddings capture the full meaning
- Retrieval is more accurate
- The model understands what each chunk refers to

---

20. Data Evolution Rule

Amended Fields

When data changes over time, capture multiple values with source and effective date:

```
{
  "customer_name": {
    "current_value": "Jane Smith",
    "history": [
      {
        "value": "Jane Doe",
        "source": "original_registration",
        "effective_date": "2020-01-15"
      },
      {
        "value": "Jane Smith",
        "source": "name_change_request_#4521",
        "effective_date": "2023-06-01"
      }
    ]
  }
}
```

Why This Matters

- Audit trails require knowing what the value was at any point in time
- Source attribution enables verification
- Effective dates enable temporal queries (“what was the value on date X?”)
- Prevents data loss when values are updated

---

21. Summary of Key Reliability Patterns

Pattern	Purpose	Implementation
Tool context pruning	Reduce noise in context	Application-side filtering
Session compression	Manage long conversations	Summarize resolved, keep active
Scratchpad	Persistent external memory	Structured markdown file
Graceful tool failure	Reliable error handling	Structured error responses

Pattern	Purpose	Implementation
Field-level confidence	Granular quality assessment	Per-field confidence scores
Structured escalation	Clean handoffs	JSON payload with key fields
Prompt caching	Cost and latency reduction	Stable content first, breakpoints
Contextual retrieval	Better RAG accuracy	Add context before embedding
Hybrid search	Better retrieval coverage	Vector + BM25 + RRF
Reranking	Precision improvement	LLM post-processing of results